FH Salzburg
MultiMediaTechnology

Basics Game Development

Asset Handling, Rendering Pipeline - Lab

Technology
Health
Media

10 Minute Quiz

Today's Topics

- Mouse / Controller Input
- Sprite Animations
- Bezier Curves

Input – Mouse - Poll

```
// Get the left Mouse Button press
if(Mouse.IsButtonPressed(Mouse.Button.Left))
{
    Console.WriteLine("Pressed Mouse Button Left");
}

// Get the mouse position relative to the window
Vector2i localMousePos = Mouse.GetPosition(window);

// Get the global mouse positon relative to the desktop
Vector2i globalMousePos = Mouse.GetPosition();
```

- https://www.sfml-dev.org/tutorials/2.5/window-inputs.php

Input – Mouse – via Callback

```
window.MouseButtonPressed += OnMouseClick;
```

```
1 reference
private void OnMouseClick(object? sender, MouseButtonEventArgs e)
{
    if(e.Button == Mouse.Button.Left)
        Console.WriteLine("Mouse Button Left");
}
```

- https://www.sfml-dev.org/tutorials/2.5/window-inputs.php

Input - Joystick

- Joysticks are identified by their index (0 to 7, since SFML supports up to 8 joysticks)

```
// Get the number of supported buttons for joystick with idx 0
Joystick.GetButtonCount(0);

// check if the button with idx 2 is pressed on Joystick with idx 0
Joystick.IsButtonPressed(0, 2);

// get the axis positions of joystick with idx 0
float x =  Joystick.GetAxisPosition(0, Joystick.Axis.X);
float Y =  Joystick.GetAxisPosition(0, Joystick.Axis.Y);
```

```
if(Joystick.IsButtonPressed(0, 0))
{
    Console.WriteLine(" Player 0 ; pressed Joystick button 0");
}
```

- Important note: in SFML.Net you need to manually call *Joystick.Update* each frame for controllers to work

```
private void HandleEvents()
{
    window.DispatchEvents();
    Joystick.Update();
}
```

Input – Joystick - Events

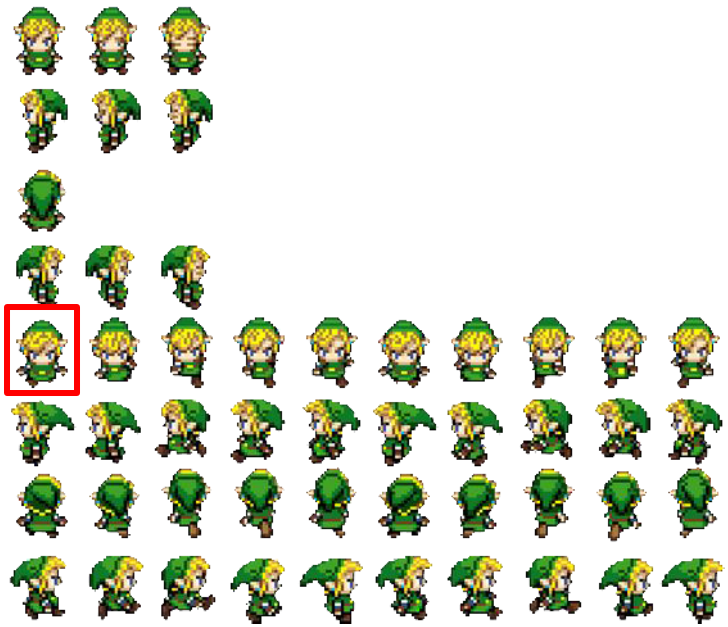
```
window.JoystickButtonPressed += OnJoystickButtonPressed;
```

```
1 reference
private void OnJoystickButtonPressed(object? sender, JoystickButtonEventArgs e)
{
    // check for Joystick with id 0
    if(e.JoystickId == 0)
    {
        // check button
        if(e.Button == 0)
            Console.WriteLine(" Player 0 ; pressed Joystick button 0");
    }

}
```

Sprite Animation

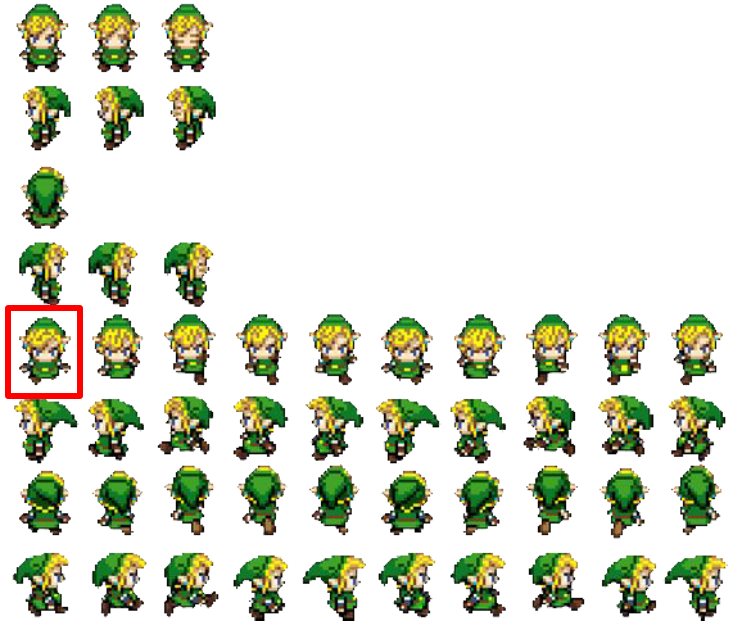
- Define the texture rect in the tiling dimension
- Define animation frame count
- Iterate over the animation frames (modified by animation speed)
- Move the the texture rect accordingly



Sprite Animation - Tiling

- Define the texture rect in the tiling dimension (example sheet 10 x 8)

```
playerSprite.TextureRect = new IntRect(0, 0,
    (int)(playerTexture.Size.X / PLAYER_TILING_X),
    (int)(playerTexture.Size.Y / PLAYER_TILING_Y));
```



Sprite Animation - Animating

- Iterate over the animation frames (modified by animation speed)

```
// Update Animationtime
animationTime += deltaTime * animationSpeed;
```

- Move the texture rect accordingly

```
int animationFrame = (int)animationTime % 3; // idle animation has 3 frames

int animationPosX = animationFrame * playerSprite.TextureRect.Width;
int animationPosY = 0; // idle animation is at y == 0

playerSprite.TextureRect = new IntRect(animationPosX, animationPosY, playerSprite.TextureRect.Width, playerSprite.TextureRect.Height);
```

Sprite Animation – additional mapping via enum

- Add a member: AnimationType currentAnimation
- Work with the enum as replacement for your animation index (just for readability)

```
if (InputManager.Instance.GetKeyPressed(Keyboard.Key.D))
{
    // Set animation type
    currentAnimation = AnimationType.RunRight;

    // Move player
    playerSprite.Position += new Vector2f(1, 0) * moveSpeed * deltaTime;
}
```

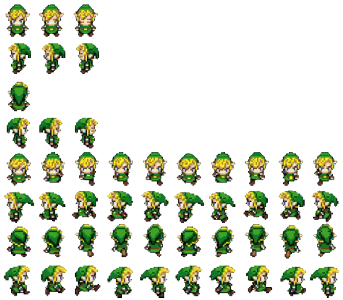
```
if (InputManager.Instance.GetKeyPressed(Keyboard.Key.D))
{
    // Set animation type
    currentAnimationIdx = 7; // run right

    // Move player
    playerSprite.Position += new Vector2f(1, 0) * moveSpeed * deltaTime;
}
```

Sprite Animation – additional mapping via enum

- Create an enum mapping the various animation states

```
private int[] animationTypeFramesCount = new int[] {
    3,
    3,
    1,
    3,
    10,
    10,
    10,
    10 };
```



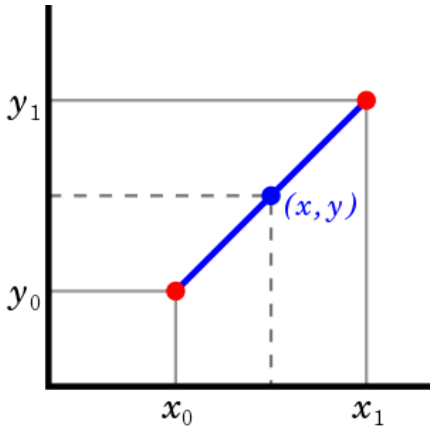
```
public enum AnimationType
{
    IdleDown  = 0,
    IdleLeft  = 1,
    IdleUp    = 2,
    IdleRight = 3,
    RunDown   = 4,
    RunLeft   = 5,
    RunUp     = 6,
    RunRight  = 7
}
```

Bézier curve

Linear Interpolations

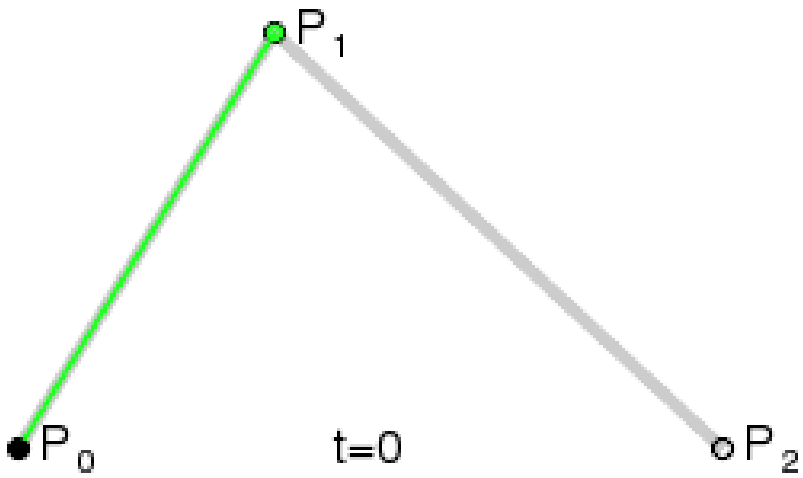
```
return firstFloat * (1f - t) + secondFloat * t;
```

```
return new Vector2f()
{
    X = Lerp(firstVector.X, secondVector.X, t, clamped),
    Y = Lerp(firstVector.Y, secondVector.Y, t, clamped)
};
```



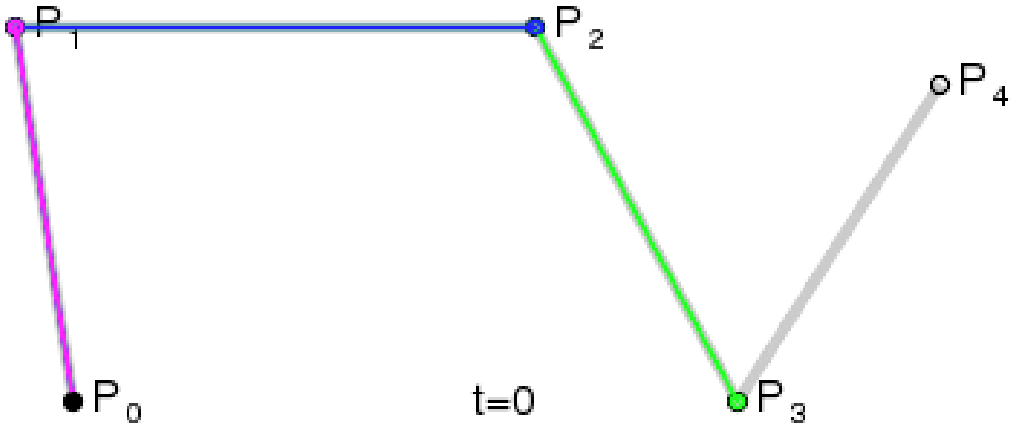
Bézier curve

- Just a series of interpolations




```
public static Vector2f EvaluateBezier(Vector2f p0, Vector2f p1, Vector2f p2, float t)
{
    var p01 = Lerp(p0, p1, t);
    var p12 = Lerp(p1, p2, t);
    return Lerp(p01, p12, t);
}
```

Higher-order curves



Sidenote: If you want a curve that crosses the points, than you need a „Catmull Rom Spline“.

FH Salzburg
MultiMediaTechnology

Technology
Health
Media

© 2025 Fachhochschule Salzburg GmbH
www.fh-salzburg.ac.at